# FLAT Module: Deterministic Finite Automata (DFA) Lexical Analyzer

## Technical Report on Formal Grammar, DFA Transition Engine, & Microcode Validation


| | |
|---|---|
| **Student Name:** Gautham Ram S | **Roll ID / Reg:** S70 |
| **Email:** gautham.ram.s70@kalvium.community | **Program:** B.Tech Computer Science & Engineering |
| **Live App:** https://integratedmicroproject.vercel.app/ | **Repository:** https://github.com/Umgauthamram/Integrated-MicroProject-FLAT-COA-DAA |


## 1. Executive Summary & Curriculum Alignment

This module implements a rigorous, custom **Deterministic Finite Automaton (DFA)** engine that serves as the front-end lexical analyzer and syntax validator for a custom RISC microcode assembly language. Aligned with **FLAT Modules 1–3 (Finite Automata, Regular Expressions, and Formal Languages)**, the engine formally models state transitions over an alphabet of instruction tokens, rejects syntactically malformed code, and translates accepted instructions into 16-bit binary control words for pipelined execution.

***Curriculum Connection:*** *Demonstrates the practical equivalence between Regular Expressions and Deterministic Finite State Machines in compiler front-end design, token extraction, and dead-state trap mechanisms.*

## 2. Mathematical Formulation & Formal 5-Tuple

Formally, the DFA is specified as a 5-tuple: **$M = (Q, \Sigma, \delta, q_0, F)$**, where:

| Component | Formal Definition & Implementation Specification |
|---|---|
| **Q (States)** | {q0, q1, q2, q3, q4, q_accept, q_reject} representing parse stages. |
| **$\Sigma$ (Alphabet)** | {LOAD, STORE, ADD, SUB, JMP, R0-R7, Imm(0-255), Comma(,), Semicolon(;)} |
| **$q_0$ (Start)** | q0: Idle initial state awaiting instruction opcode mnemonic. |
| **F (Accept)** | {q_accept}: Final state reachable strictly upon encountering semicolon terminator. |
| **q_reject (Dead)** | Dead trap state reached immediately upon any lexical or syntactic violation. |

## 3. DFA State Transition Table & Grammar Paths

The state transition function $\delta: Q \times \Sigma \rightarrow Q$ maps every configuration to a unique deterministic next state:

| State | Opcode | Reg (R0-R7) | Comma (,) | Number/Imm | Semi (;) | Other |
|---|---|---|---|---|---|---|
| q0 | q1 | q_reject | q_reject | q_reject | q_reject | q_reject |
| q1 | q_reject | q2 | q_reject | q4 (if JMP) | q_reject | q_reject |
| q2 | q_reject | q_reject | q3 | q_reject | q_reject | q_reject |
| q3 | q_reject | q4 (Reg) | q_reject | q4 (Imm) | q_reject | q_reject |
| q4 | q_reject | q_reject | q_reject | q_reject | q_accept | q_reject |
| q_acc | q_reject | q_reject | q_reject | q_reject | q_reject | q_reject |
| q_rej | q_reject | q_reject | q_reject | q_reject | q_reject | q_reject |

## 4. 16-Bit Micro-Instruction Control Word Encoding

Upon reaching **q_accept**, the DFA synthesizes a 16-bit binary control word formatted as follows:

**[15:13] Opcode (3b) | [12:10] Dest Reg (3b) | [9:7] Source Reg (3b) | [6:0] Immediate / Offset (7b)**

- **LOAD**: 001 | **STORE**: 010 | **ADD**: 011 | **SUB**: 100 | **JMP**: 101
- **Registers R0–R7**: 000 through 111 | **Signals**: RegWrite, ALUSrc, ALUOp, MemRead, MemWrite, Branch

## 5. Verification Suite & Rejection Cases

| Input Statement | DFA State Path | Verdict | Synthesized Hex Word / Error |
|---|---|---|---|
| ADD R1, R2; | q0 → q1 → q2 → q3 → q4 → q_accept | ACCEPTED | 0x6480 (Op:ADD, Rd:R1, Rs:R2) |
| LOAD R3, 25; | q0 → q1 → q2 → q3 → q4 → q_accept | ACCEPTED | 0x2C19 (Op:LOAD, Rd:R3, Imm:25) |
| JMP 128; | q0 → q1 → q4 → q_accept | ACCEPTED | 0xA000 (Op:JMP, Imm:0) |
| ADD R1 R2; | q0 → q1 → q2 → q_reject | REJECTED | Syntax Error: Missing comma |
| SUB R1, 10 | q0 → q1 → q2 → q3 → q4 → q_reject | REJECTED | Syntax Error: Missing semicolon terminator |
| MULT R1, R2; | q0 → q_reject | REJECTED | Syntax Error: Invalid opcode MULT |

## 6. Conclusion & Inter-Module Linkage

The FLAT DFA module guarantees 100% lexical and grammatical soundness before execution, preventing malformed control words from entering the CPU pipeline. Validated control words pass directly into the COA 5-stage pipeline simulator.